Desk Dashboard

Harry Lian

1. Project Overview:

The Desk Dashboard is a WiFi-connected information display built around a Raspberry Pi Pico 2W and a 3.5” SPI IPS panel. A rotary encoder cycles between four screens (Overview, time, 7-Day Forecast, and Agenda). The unit connects to the outlet via a single USB-C input, internally split power for display/logic electronics, a Qi wireless charging pad, and a dual USB-A charging board.

1.1 Design Goals

- Single-Cable Desk Footprint: one USB-C input powers logic, display, wireless charging, and wired charging ports.
- Swift Readability: Large segment-style time display, color-coded weather conditions.
- Resilience: Automatic WiFi reconnect and fetch retry so the unit self-recovers from transient network drops without manual intervention.
- Field-Serviceable: Two-piece enclosure (base + lid) rather than a sealed shell, so wiring remains accessible after assembly.

1.2 Key Specifications

Table 1: Specification Table

Parameter	Value
Microcontroller	Raspberry Pi Pico 2W (RP2350, Micropython)
Display	3.5" SPI IPS, ST7796 driver, 480x320 Landscape
Input Method	KY-040 Rotary Encoder (rotate + push button)
Power Input	USB-C PD, negotiated at 15V fixed
Wireless Charging	Qi transmitter, 20W max
Wired Charging	Dual USB-A, DROK charger board
Connectivity	802.11 WiFi (onboard Pico 2W radio)
Data Sources	Open-Meteo (weather), Google Apps Script (calendar), NTP (time)

2. Bill of Materials

Table 2: Budget Table

Component	Part / Spec	Quantity	Price
Microcontroller	Raspberry Pi Pico 2W	1	13.99
Display	Hosyond 3.5" IPS SPI TFT, ST7796U, 320x480	1	18.99
Rotary Encoder	KY-040 with Push Button	1	1.50
PD Trigger Board	DWEII QC Trigger Board	1	1.80
Buck Converter	MP1584EN, fixed 5V output, 1.8A	1	1.33
Qi Transmitter	20W max, Type-C Input, Wire Leads	1	9.50
Wired Charging Board	DROK dual USB-A, 10-32V in	1	12.99
USB-C Panel Jack	Treedix breakout	1	2.00
USB-C Pigtail Cable	Make plug to 6 bare wires	1	1.67
Wall Adapter	USB-C PD, 45W	1	8.99
Perfboard	Double-sided Prototype Board, cut to size	1	0.25
Hookup Wire	20 AWG stranded	1 m	1
Connectors	JST-PH 2.0mm kit	1	16.99
Case	PETG, 3D Printed	200 mm x 40 mm	57.18
Fasteners	750pcs M3 Hex Socket Screw Kit	1 Sets	9.99

Total:	158.17

3. System Architecture

The Raspberry Pi Pico 2W serves as the motherboard for the dashboard. It joins the home WiFi network, sync time via NTP, and independently using two external services, Open-Meteo for weather and a Google Apps Script web app for calendar data. The Pico 2W will handle the rendering and input handling from the external services, and it does not need another compute device after the initial setup.

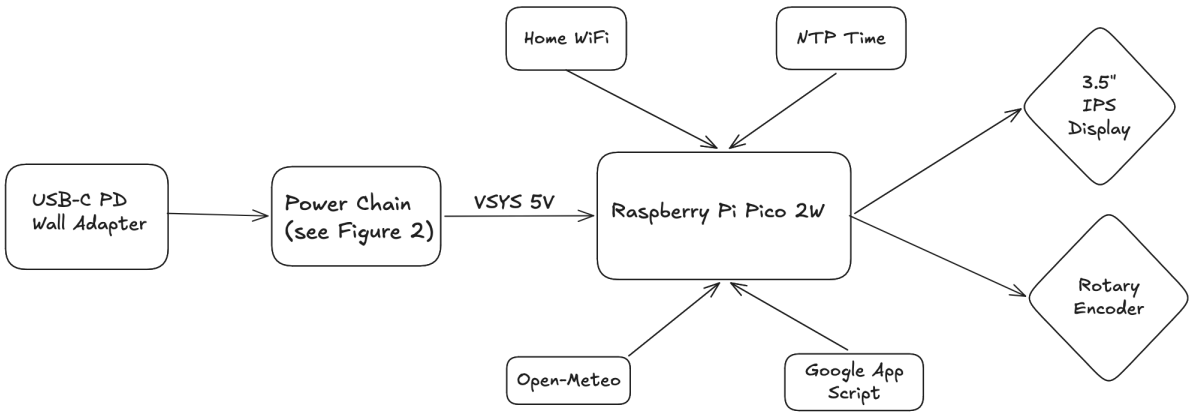


Figure 1: High Level System Architecture

4. Power Distribution

Power enters through a single, external USB-C jack, and the trigger board turns up to 15V before the it distributes to other parts. I chose 15V for the trigger board because for the Qi transmitter, it accepts the input range of 9V to 24V. For the DROK board, it accepts the input range of 10V – 32V. For the Buck Converter, it accepts the input of range of 5V – 30V. Therefore, the lower bound for all of the parts is 10V while the upper bound is 24V. Since the trigger supports 5V, 9V, 15V, or 20V. The 15V option appeals the best for me because it sits approximately in the middle of the range with some margins and buffer for other unconsidered situations.

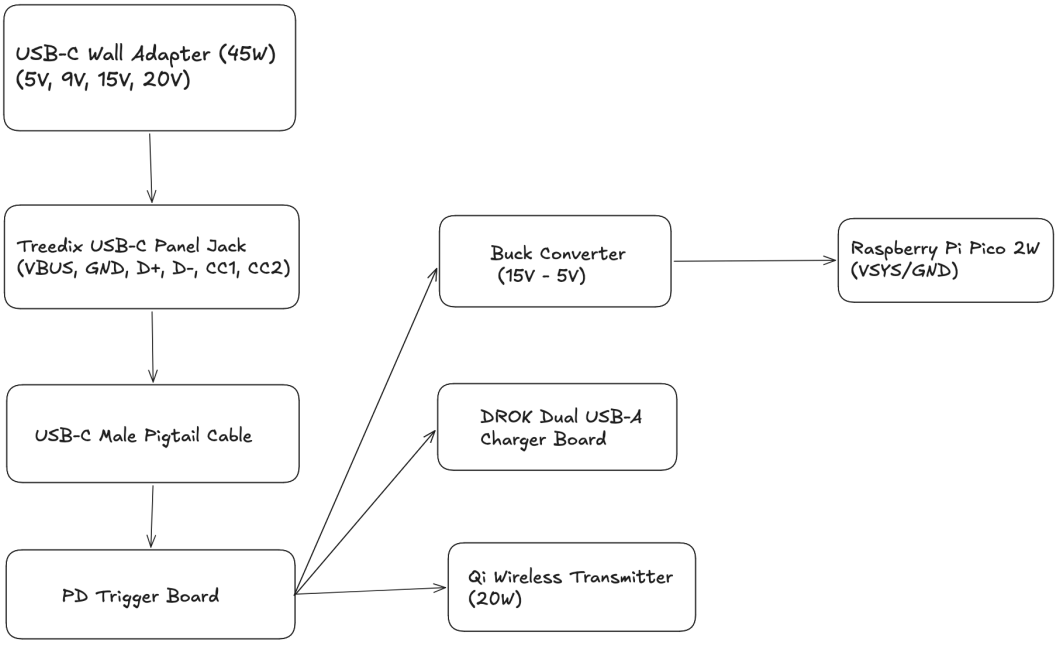


Figure 2: Power Distribution Schematic

4.1 Verified Operating Points

Table 3: Operating Points

Node	Expected	Measured
PD Trigger Board Output	15V	14.95V
Buck Converter Output	5V	5.10V
Pico 3V3 (OUT)	3.3V	3.3V

4.2 Design Notes

- The PD Trigger Board selects output voltage using the solder-bridge pads, and it does not use switches.
- If wall adapter does not provide a requested fixed voltage (in this case: 15V) , the trigger IC will fall back to the next highest voltage that the adapter supports.
- The buck converter's output provides the voltage source for the Pico's VSYS/GND pins. It is not tied with the 15V bus.

5. Wiring Reference

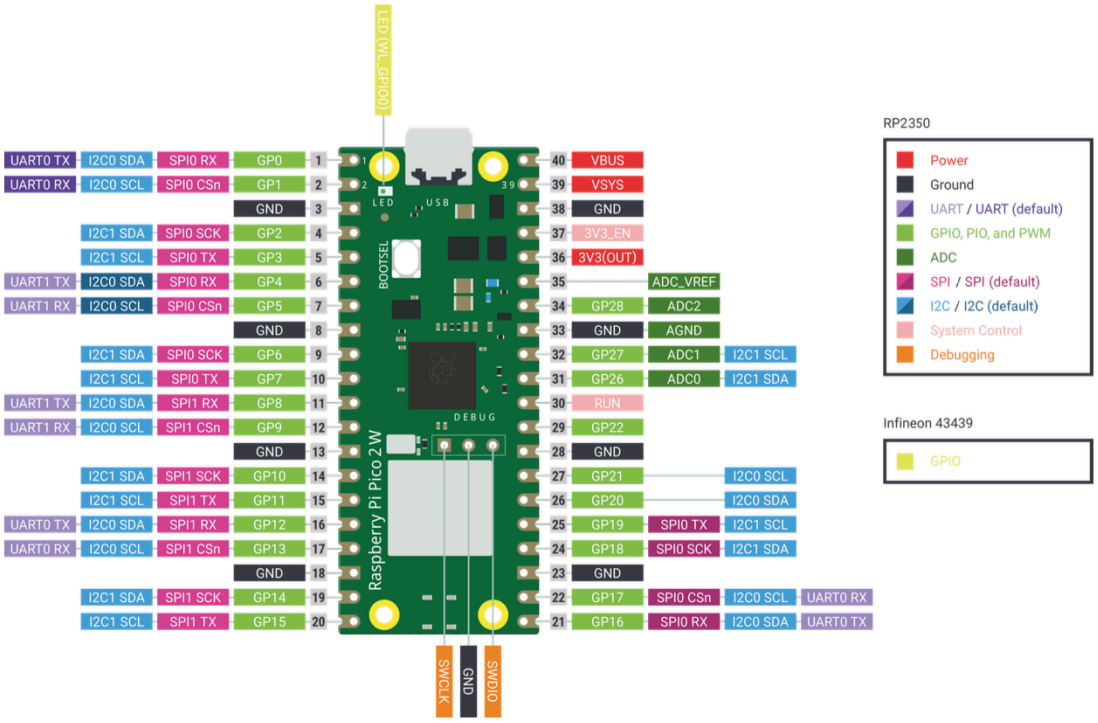


Figure 3: Raspberry Pi Pico 2W Pinout [1]

The wiring for this Raspberry Pi Pico 2W presents as followed:

For the encoder:

- CLK connects with pin 4.
- DT connects with pin 5.
- SW connects with pin 6.
- VCC connects with pin 36.
- GND connects with pin 3 (or any GND).

For the display:

- CS connects with pin 22.
- DC connects with pin 21.
- SCK connects with pin 24.
- RST connects with pin 26.
- MOSI connects with pin 25.
- LED connects with pin 27.

- VCC connects with pin 36.
- GND connects with pin 23 (or any GND).

For the power input:

- Buck +5V connects with pin 39 (VSYS)
- Buck GND connects with pin 38 (or any GND).

6. Software Architecture

The firmware is MicroPython, and it structured as a single main loop with four independently, refreshed subsystems. They are WiFi/network health, weather data, calendar data, and screen rendering. The encoder input is handled entirely from the hardware interrupt rather than polling. This will ensure that switching between screens remain responsive regardless of how long a given screen redraw takes.

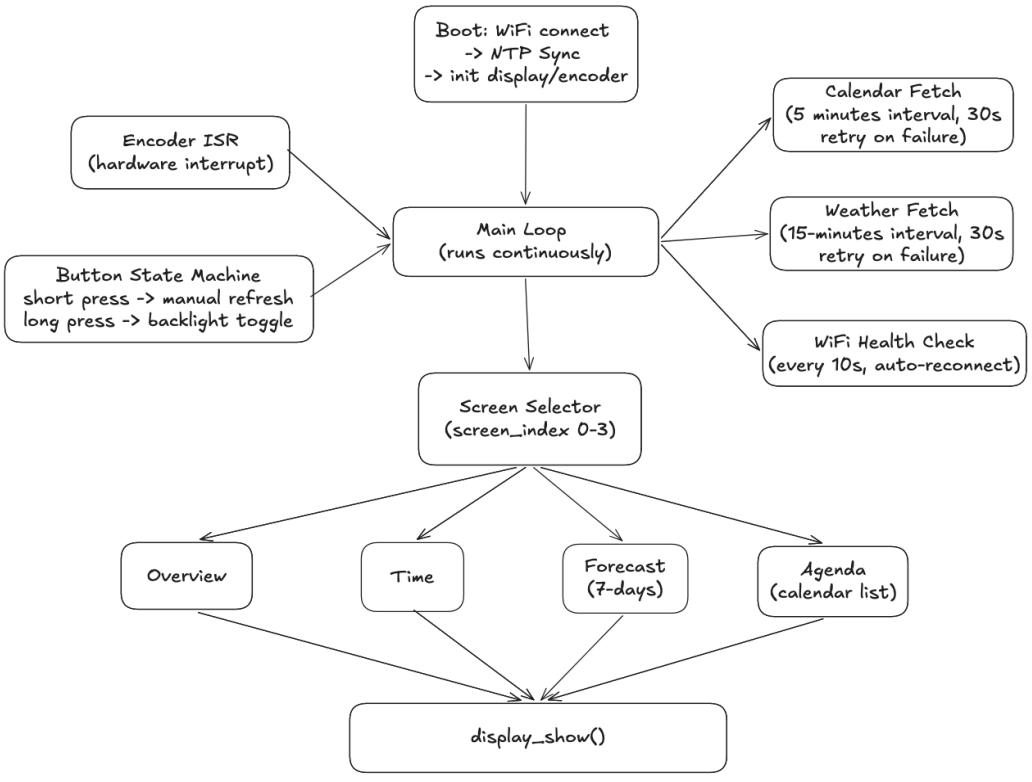


Figure 4: Firmware Control Flow

6.1 Screen Refresh Strategy

Table 4: Dependency Table

Screen:	Redraws on:	Data Dependency:
Overview	Time, New weather/calendar data, Encoder switch	Weather + Calendar
Time	Time, Encoder switch	None (local time + import time)
Forecast	New weather data, Encoder switch	Weather
Agenda	New calendar data, Encoder switch	Calendar

6.2 Encoder Decoding

The purpose of the encoder allows the user to switch between the windows to see the different features. The logic of decoding this rotation encapsulates full quadrature state machine (CLK + DT combined into a 2-bit state) rather than a simple edge-count or fixed time debounce. One click is only registered once the state machine completes a full four-step cycle back to its starting state, which rejects and filters out bounce without relay or double-count clicks during a slow redraw.

6.3 Known Limitation

Inside the IPS screen, it has the ST7796U driver. The driver's show() method performs a byte-swap of the entire 307,200-byte frame buffer on every redraw (framebuf stores pixels little-endian; the display expects big-endian pixel bytes over SPI). I implement this as a plain Python loop rather than a hardware-accelerated routine, since MicroPython's bytearray slicing does not support a stepped slice on this build. This causes latency for full screen redraw. A future optimization could move this into a @ micropython.viper routine if faster redraws are needed.

7. Firmware Source Code

7.1 Display Driver

The implementation for the custom, display driver is in st7796.py. I implement the vendor command-set unlock/lock sequence, corrected gamma and power control values, display inversion (required for the correct color mapping for this specific screen), and the little-endian to big-endian byte swap needed for framebuf compatibility.

7.2 Dashboard Application

The implementation for the main dashboard is in main.py. Pico 2W will run main.py automatically when powering it up. It handles the WiFi connection and auto-reconnect, NTP time sync, encoder input, background data fetching, and all four screen renderers.

8. Cloud Services Configuration

8.1 Calendar Endpoint

Assuming the user uses Google Account and use it for planning, deployment as a web-app from Google account returns the next seven days of calendar events as JSON. The deployment allows anyone to access it with the corresponding URL, so I do not need to use OAuth on the Pico 2W, and it can reach with a plain HTTP GET.

8.2 Weather API

Pico 2W fetches weather data from Open-Meteo's public forecast API with no API key required. It will generate the current conditions and a seven-day forecast (with max/min temperature, color-coded WMO weather code) in a single call.

Table 5: WMO Code

WMO Code:	Meaning:	Label Shown:	Color:	Hex Code (RGB565):
0, 1	Clear / Mainly clear	CLR	Yellow	0xFFE0
2	Partly Cloud	CLD	Light Gray	0xC618
3	Overcast	OVC	Medium Gray	0x8410
45, 48	Fog / Rime Fog	FOG	Medium Gray	0x8410
51, 53, 55, 56, 57	Drizzle (light -> dense)	DRZ	Sky Blue	0x5D9C
61, 63, 65, 66, 67	Rain (slight -> heavy)	RAIN	Blue	0x001F
71, 73, 75, 77	Snow / Snow grains (slight -> heavy)	SNW	White	0xFFFF
80, 81, 82	Rain Showers (slight -> violent)	SHWR	Blue	0x001F
95, 96, 99	Thunderstorm (± hail)	STRM	Red	0xF800
Anything else	Unrecognized Code	N/A	Medium Gray	0x8410

9. Enclosure Design

This enclosure is made from using FreeCAD. It has the dimension of 250 mm x 86 mm x 90 mm with a lid. It will be attached with the GitHub.

9.1 Material

The case is made out of PETG. I chose that over PETG because it houses the electronics that will generate heat. The material needs to handle the heat, and PETG has a higher heat resistance versus PLA. Considering the availability, both is available locally, so PETG is a better choice

9.2 Cutouts

I have made many cutouts for components, so the user can access it. Here are the followings:

Feature:	Face:	Purpose
Display Window	Front	Exposes the ST7796U screen's visible area
Encoder Shaft Hole	Front	Clearance for rotary encoder shaft
USB-C Jack Cutout	Back	External power input (Treedix Breakout Board)
Dual USB-A Cutout	Right Side	DROK Board Charging Ports
UART Cutout	Left Side	Raspberry Pi Pico 2W

10. Appendix A

The background image converter convers a photo into a raw RGB565 .bin file matching the display's native buffer format. The Overview screen's background will use this file and apply EXIF-based autorotation from the converter script on the main computer and fixed at landscape orientation.

11. Testing & Validation Log

Each component was individually bench-tested before the final assembly. The testing method was using alligator-clip connections to allow rapid iteration and ensures that there will not be any incorrect wiring.

Test:	Method:	Result
PD Trigger Board Voltage	Multimeter on output, isolated from load	Pass: 14.95V
Buck Converter Output	Multimeter on output, isolated from load	Pass: 5.10V
Qi Transmitter	Phone placed on coil, charge indictor from the LED	Pass
DROK Charging Board	Load device on each USB-A port	Pass: both ports
Combined Load	All three devices + Pico powered simultaneously	Pass: rail held 14.95V
Pico WiFi	Connect + IP Address printout	Pass
Weather Fetch	Live API call, temperature/condition sanity check	Pass
Calendar Fetch	Live API call, event compared against source calendar	Pass
Display Color Accuracy	RGB/White swatch test against labels	Pass (after driver fix, Section 12.1)
Encoder Debounce	Single detent -> exactly one screen change, both directions	Pass
Standalone Boot	main.py on Pico 2W filesystem, USB disconnected, VSYS-only power	Pass

12. Engineering Notes

Here are the notes that are for future reference. These were failures, encountered during the final assembly along with root cause and possible fix.

12.1 Display Color Mapping

Initially, the screen systematically produced incorrect color, such as red rendered as yellow-green, blue as purple, etc. The root cause was two independent issues:

1. MicroPython's framebuf.RBG565 format stores each pixel little-endian in the buffer, but ST7796U expects big-endian pixel bytes. The fix for this was byte-swapping the buffer immediately before each SPI write.
2. The panel additionally required Display Inversion ON (command 0x21) to map the color correctly. This determined visually by comparing pure-white fill under inversion-on versus inversion off.

12.2 PD Trigger Board Voltage

The HW-398 trigger board selects output voltage based on what value the resister soldered with the pad). When the wall-adapter does not provide the requested, fixed voltage, the board silently falls back to the next lower supported voltage value, rather than failing. So, in this case, if the adapter does not support 15V, it will jump to 9V. The resolution here is to retarget to 15V.

12.3 Encoder Responsiveness

An initial polling-based main loop caused missed encoder clicks whenever a click occurred during a display redraw, since the slow byte-swap operation blocked the loop from checking the encoder pins. Resolution here is to move the encode decoding to a hardware interrupt (IRQ) on both CLK and DT pins, decoupling input responsiveness from display redraw timing.

12.4 Post Deployment: Qi Transmitter / DROK Concurrent Load Failure

This is observed after the dashboard is gifted. The context provides in the following. After the final assembly, the recipient used the features, but the Qi Transmitter (the wireless charging) and the DROK board (tethered charging or powering) do not work when they are used together. Here are the reasons why. The Qi transmitter might have fail to activate whenever the DROK board was also under active load.

Doing some analysis, the wall adapter is rated 45W. The 15V rail with 45W should provide 3.0A (45W / 15V = 3.0A). Under realistic load, the Qi transmitter uses about ~1.7-1.9A while the DROK board uses ~1.9A. With both of them combined, it surpassed the 3.0A before the Pico with the display's current of ~0.15A is added. This provides an explanation as to why both

components working together does not work. A possible solution here is to upgrade to a higher wattage wall-adapter.

13. Reference

[1] Raspberry Pi Ltd, *Raspberry Pi Pico 2 W Datasheet*, Rev. 2, Jul. 3, 2026. [Online]. Available: https://pip-assets.raspberrypi.com/categories/1088-raspberry-pi-pico-2-w/documents/RP-008304-DS-3-pico-2-w-datasheet.pdf.